
Recycling the Living: Dormant-Neuron Interventions Do Not Work by Reviving Dead Units

Nikolaos Mavros
University of Thessaly

nmavros@uth.gr

Konstantinos Kolomvatsos
University of Thessaly

kostasks@uth.gr

Abstract

A neural network can lose more of its ability to learn than any other configuration we measure — 13.0 percentage points of online accuracy across 200 tasks — while every published definition of a dead neuron reports that exactly 0.00% of its units are dead. That is not a corner case; it is a tanh network at its calibrated learning rate, and it is the sharpest of six independent demonstrations that the field’s death metrics do not measure one underlying thing. We then ask whether the metric’s target is the mechanism it is credited with. Sweeping ReDo’s dormancy threshold τ from 0 to 0.25 on online permuted MNIST (190 runs, 10 seeds), the genuinely dead share of what gets recycled falls from 31.4% to 12.3% while final accuracy moves by 0.169 percentage points, 3.6% of the benefit of recycling at all. A control matched in *size* — k recomputed per layer at every event, not a fixed percentage — recovers 88–92% of ReDo’s benefit while recycling sets that are 87–91% alive; recycling the *highest*-scoring units still recovers two thirds. Targeting is real and small. Two anomalies reported inside the *Nature* paper on plasticity replicate: L_2 at its beneficial dose nearly doubles dead units, and online normalisation carries 38% more of the units it was designed to prevent while being the best arm we measure. And 21.7% of dead units revive within one task with no intervention at all. We close with five concrete changes to measurement practice.

1 Introduction

Train a network on a stream of tasks and it will, eventually, stop being able to learn new ones. Alongside that decline, a growing fraction of its units stop firing. The two curves are easy to plot together, they move together, and the field has largely read the second as the cause of the first. Roughly fifteen mitigation methods now rest on that reading. The most direct of them, ReDo (Sokar et al., 2023), periodically identifies units whose activation magnitude has fallen below a threshold and re-initialises them; Continual Backpropagation (Dohare et al., 2024) continually reinitialises low-utility units; a family of resetting, regularisation and reinitialisation methods follows the same logic (Nikishin et al., 2022; Kumar et al., 2023; Elsayed & Mahmood, 2024; Hernandez-Garcia et al., 2025).

The evidence against that reading is already in print, in three places, and has never been assembled. Sokar et al. (2023) report that in a sufficiently over-parameterised regime, *pruning* the dormant units costs nothing and ReDo buys nothing — if the units contributed something recoverable, removing them should hurt. Lyle et al. (2023) report plasticity loss occurring in the *absence* of saturated units. Kooi et al. (2024) report that tanh networks have *fewer* dormant units, *higher* effective rank, and *worse* performance, reversing the sign of the correlation the whole programme depends on. The field’s own survey (Klein et al., 2024) lists dormant and saturated units as one hypothesised cause among nine, concludes that generic regularisation outperforms interventions built specifically to target dormancy, and names understanding mechanism as the highest-priority open problem.

This paper is the systematic version of that scattered evidence. We do not propose a method. We build one instrument carefully, apply it to the methods that already exist, and report what it reads.

Scope: supervised continual learning, not RL. ReDo was designed and validated in deep reinforcement learning. Our evidence comes from continual supervised learning — online permuted MNIST, the protocol of the *Nature* paper this work engages most closely (Dohare et al., 2024), plus a small convolutional and an activation-function arm. This is a deliberate choice rather than a limitation to be excused later. The mechanism at issue — that recycling helps *because* it restores dead units — is a claim about networks, not about environments, and it is testable far more tightly where the data distribution is under experimental control, the outcome measure is not itself a stochastic return, and ten seeds cost hours rather than weeks. It is also where the original authors’ own negative result points: Sokar et al. (2023) found no ReDo gain in the over-parameterised low-dimensional regime, which is an invitation to test the mechanism somewhere its effect can be resolved. Where our results diverge from theirs we say so and treat the divergence as a domain difference, not as a correction.

Contributions.

1. **ReDo’s benefit does not require targeting dead units.** As τ sweeps from 0 to 0.25, the genuinely dead share of the recycled set falls from 31.4% to 12.3% while late-window accuracy rises by 0.169 *pp* [0.143, 0.196], 3.6% of the +4.76 *pp* that recycling buys over no intervention. A control matched in *size* — not percentage; k is recomputed per layer at every event — recovers 88–92% of ReDo’s benefit while recycling sets that are 87–91% alive. Targeting contributes something real and small; the strong form of the causal hypothesis is refuted.
2. **The field’s death metrics disagree by a factor of 3.5 on identical activations,** and the disagreement is systematic rather than noisy: each definition’s blind spot traces to an identifiable design choice, and under tanh three of the four definitions report exactly zero on a network that has lost 13.0 *pp* of plasticity.
3. **Two anomalies inside Dohare et al. (2024) replicate and are given a mechanism.** L_2 regularisation at its beneficial dose improves accuracy by +1.50 *pp* while raising dead units from 20.7% to 40.2% and cutting effective rank from 100 to 50; online normalisation (Chiley et al., 2019) is the best-performing arm we measure (+3.16 *pp*) and carries 38% more dead units than plain backpropagation.
4. **A fifth of dead units revive with no intervention at all.** On a fixed reference distribution, a unit that is exactly zero at one task boundary is firing again at the next with probability 21.7% [21.4, 22.2], and 87.3% of all units die at least once against a steady-state prevalence of 26%. Death is a state units pass through, which undercuts the premise — shared by every mitigation method surveyed — that dead units are permanently lost capacity requiring active restoration.

Roadmap. Section 2 positions the work. Section 3 defines the four death metrics formally, and does real argumentative work: the reader needs to hold them apart before seeing the numbers. Section 4 gives the setting, the pre-registration procedure, and the size-matched control design. Section 5 reports one subsection per claim, including a pre-registered hypothesis that did not confirm. Section 6 turns the findings into five changes to measurement practice. Sections 7 and 8 discuss and bound the result.

2 Related work

Five literatures, one phenomenon. Inactive units are studied under at least five names by communities that largely do not cite each other. In activation-function design the problem is *dying ReLU*: a unit whose pre-activation is negative for every input emits zero, receives zero gradient, and cannot recover through the gradient path (Glorot et al., 2011; Maas et al., 2013; Lu et al., 2020). In deep RL it is the *dormant neuron phenomenon*, a symptom of non-stationary targets to be fixed by recycling (Sokar et al., 2023). In

continual learning it is one face of *loss of plasticity*, argued to require a non-gradient source of randomness to sustain learning (Dohare et al., 2021; 2024). In interpretability it is an observed structural property of trained language models, with more than 70% of neurons in some layers of a 66B model never activating (Voita et al., 2024), and its extreme form in sparse autoencoders is the *dead latent* (Gao et al., 2024). And in pruning it is a *resource*: Dufort-Labbé et al. (2025) deliberately promote death in order to harvest structured sparsity from it, while Whitaker & Whitley (2023) revive dead units by pruning their offending incoming connections rather than resetting the unit.

What is actually contested. These communities disagree in print about whether inactive units *cause* degraded learning, merely accompany it, or are beside the point. Sokar et al. (2023) report both that recycling dormant units helps and that pruning them is free, and that ReDo gives no gain when the network is sufficiently over-parameterised. Lyle et al. (2023) connect plasticity loss to loss-landscape curvature and observe it without saturated units. Lyle et al. (2024) decompose it into several independent mechanisms on which single-target interventions are individually insufficient. Kumar et al. (2021) and Gulcehre et al. (2022) identify collapse of the feature matrix’s effective rank as a distinct pathology, the latter partly deflationary about how much bootstrapping explains. Kooi et al. (2024) show the dormancy–performance correlation reversing under a change of activation function. Lewandowski et al. (2024) locate the cause in the curvature of the loss surface instead, and propose a metric — the average sign entropy of pre-activations — that is not a death count at all. Klein et al. (2024) survey roughly fifty mitigation methods and conclude that generic regularisation beats dormancy-targeted intervention.

The mitigation families themselves are worth separating, because they make different implicit claims about the pathology. Resetting methods — ReDo, Continual Backpropagation, primacy-bias resets (Nikishin et al., 2022) — assume the affected units are recoverable. *Adding* capacity rather than recovering it, as in plasticity injection (Nikishin et al., 2023), assumes they are not. Architectural changes such as concatenated ReLU (Abbas et al., 2023) assume the problem can be designed out of the activation function. And Berariu et al. (2021) question whether the capacity was ever lost in the sense these methods presume. Our results speak most directly to the first family, and Section 5.4 bears on the premise all four share.

The definitions themselves. Less attention has gone to whether the quantity being measured is stable. Sokar et al. (2023)’s τ -dormancy normalises a unit’s mean absolute activation by its layer’s mean, making it a *relative* measure; Dohare et al. (2024) count units that are exactly zero, an *absolute* one. The two are used interchangeably in citing work. Kooi et al. (2024) give the one published demonstration we are aware of that the choice matters: under tanh, a dormant unit is not silent but pinned at a nonzero constant, so the dormancy count understates the damage. Section 5.2 generalises that observation into a systematic comparison of four definitions computed simultaneously on identical activations.

Positioning. The closest work to ours in spirit is Lyle et al. (2024), which separates mechanisms by intervening on them individually, and Klein et al. (2024), whose formalism of an intervention operator over parameters is the right frame for a causal question about resetting. Neither runs the specific comparison this paper is built around: a recycling control matched in cardinality to the dormant set rather than in percentage. Sokar et al. (2023)’s own random baseline resets a fixed fraction of units on a cosine schedule, which varies *how many* units are reset together with *which* ones. Separating those two is the experimental core of what follows.

3 Four definitions of a dead neuron

Let $h_i^\ell(x)$ be the post-activation output of unit i in hidden layer ℓ on input x , and let $\mathcal{B}$ be a fixed probe batch of 2048 examples. Write H^ℓ for the number of units in layer ℓ . All four definitions below are computed on the same forward pass over the same $\mathcal{B}$, so any disagreement between them is a property of the definitions and not of sampling.

Exactly dead. Following Dohare et al. (2024),

$$\text{DEAD-EXACT}(i, \ell) = \mathbb{I}[h_i^\ell(x) = 0 \text{ for all } x \in \mathcal{B}]. \quad (1)$$

Table 1: The four definitions, their sources, and the specific failure mode of each. Every entry in the third column is measured in Section 5.2.

Definition	Criterion	Source	Blind to
DEAD-EXACT	output is exactly 0 on every probe input	Dohare et al. (2024)	Any smooth activation. Under SiLU it is identically zero by construction; under GELU it fires only through a float32 underflow, so the <i>same unit</i> can be dead in one precision and alive in another. Under tanh it is zero while the network is saturating.
DORMANT $_{\tau}$	mean $ h $, divided by the layer mean, is $\leq \tau$	Sokar et al. (2023)	A layer whose activations all shrink uniformly: the ratio is preserved, so dormancy reads near 0% on a functionally silent layer. Conversely, a few very-high-magnitude units make healthy neighbours look dormant.
DEAD-ABS $_a$	mean $ h $ is below a fixed a	this work, as a control	Scale. A layer whose activations are legitimately small is flagged. The threshold has no principled value, which is why we report three.
SATURATED	pinned within ϵ of a bound on every input	Kooi et al. (2024)	Unbounded activations, where it is undefined — which is every activation used by the four source papers this work engages.

The comparison is to exact floating-point zero, with no tolerance. This is the definition in the *Nature* paper and the one most often meant by “dead”.

τ -dormant. Following Sokar et al. (2023), define the score

$$s_i^{\ell} = \frac{\mathbb{E}_x |h_i^{\ell}(x)|}{\frac{1}{H^{\ell}} \sum_{k=1}^{H^{\ell}} \mathbb{E}_x |h_k^{\ell}(x)|}, \quad \text{DORMANT}_{\tau}(i, \ell) = \mathbb{I}[s_i^{\ell} \leq \tau]. \quad (2)$$

The denominator is the layer’s own mean, which makes this a *relative* measure: it asks whether a unit is quiet *compared to its neighbours*, not whether it is quiet. ReDo selects its recycling set with s_i^{ℓ} , using a 64-example batch by default. We evaluate $\tau \in \{0, 0.01, 0.025, 0.05, 0.1, 0.25\}$. Note that DORMANT $_{\tau}[0]$ and DEAD-EXACT coincide by construction, which we use as a cross-implementation check.

Absolutely dead. As a control for the normalisation above we add, for a fixed threshold a ,

$$\text{DEAD-ABS}_a(i, \ell) = \mathbb{I}[\mathbb{E}_x |h_i^{\ell}(x)| < a], \quad a \in \{10^{-6}, 10^{-4}, 10^{-2}\}. \quad (3)$$

No layer normalisation appears. This is the only one of the four that can see a layer whose activations have all shrunk uniformly toward zero.

Saturated. For bounded activations only, with extremes e and tolerance $\epsilon = 10^{-3}$,

$$\text{SATURATED}(i, \ell) = \mathbb{I}\left[\min_e |h_i^{\ell}(x) - e| < \epsilon \text{ for all } x \in \mathcal{B}\right]. \quad (4)$$

For unbounded activations this returns *undefined*, never 0. The distinction matters: reporting 0 would let a reader conclude that a ReLU network has no saturated units, when in fact the question does not apply.

Table 1 states what each definition is blind to. These are not hypothetical: Section 5.2 measures every row.

Effective rank. Reported alongside the death counts throughout, because a network can lose representational capacity without any unit reaching zero. We use the *entropy-based* definition of Roy & Vetterli (2007):

given the singular values $\sigma_1 \dots \sigma_q$ of the layer’s activation matrix on the probe batch and $p_k = \sigma_k / \|\sigma\|_1$,

$$\text{erank} = \exp\left(-\sum_k p_k \log p_k\right), \quad (5)$$

with $p_k \log p_k := 0$ where $p_k = 0$. This is the quantity Dohare et al. (2024) report, and it is *not* the thresholded srnk_δ of Kumar et al. (2021); the two are often both called “effective rank” and are not interchangeable, so numbers here should not be compared against srnk_δ values in the RL literature.

Two probe distributions. Every quantity above is computed twice: once on a batch drawn from the *current* task’s distribution, and once on a *reference* batch fixed for the entire run (the unpermuted images). A unit that is silent on the permutation currently being trained on may fire perfectly well on a distribution the network has not seen recently, and vice versa. Without the second probe these two situations are indistinguishable. None of the source papers uses one; Section 5.2 shows they differ by enough to matter.

4 Experimental setup

Three things in this section carry more weight than the rest, and a reader short of time should read those: the size-matched control design in Section 4.2, which is what makes C1 testable at all; the pre-registration procedure in Section 4.3, which fixes what counts as a result before any data exists; and the batch-size calibration story below, which is the one place where we knowingly used a second-best fix and where a reader should discount accordingly.

4.1 Setting

Task stream. Online permuted MNIST, following Dohare et al. (2024): 200 tasks, each a fresh fixed permutation of the input pixels, each seen for exactly one pass over the 60,000 training images. The outcome measure, fixed in advance, is *online accuracy* — the fraction of training examples classified correctly on the forward pass taken *before* the parameter update for that minibatch, averaged over the task. Held-out probe accuracy on a fixed evaluation batch is logged alongside as a secondary measure.

Model. A 784–500–500–500–10 ReLU (Nair & Hinton, 2010) MLP with Kaiming uniform initialisation (He et al., 2015), trained with SGD and momentum 0.9 at batch size 128 — 469 steps per task, 93,800 steps per run. Width 500 rather than Dohare et al.’s 2000 because they report plasticity loss to be most pronounced at smaller widths. Activation function and normalisation are configuration fields, which is what makes Section 5.2’s activation sweep a one-field change rather than a reimplementaion. Figure 1 shows the setting together with where the instrument reads.

Calibration, and an honest methods anecdote. Our first configurations showed too little plasticity loss to study: at $\text{lr} = 0.01$ the drop from the early to the late window was 1.33 *pp* against the 3 *pp* the pre-registered reproduction gate demanded. The reason is arithmetic rather than mysterious. Dohare et al.’s online permuted MNIST is genuinely online — batch size 1, one example at a time — so their setup performs $800 \times 60,000 \approx 48\text{M}$ updates where ours performs $200 \times (60,000/128) \approx 94\text{k}$, a factor of 512 fewer. Plasticity loss accumulates per update, not per task boundary.

The correct fix is batch size, and we did not take it: batch size 1 at 200 tasks does not fit the 11-hour job ceiling of the compute we had. We instead followed the pre-registered failure response, which specifies raising the learning rate first, and climbed a ladder of five values. The drop is perfectly monotonic in step size across two decades (−1.18 *pp* at $\text{lr} = 0.001$ to +4.54 *pp* at $\text{lr} = 0.1$; Table 7), which is Dohare et al.’s own finding — largest step size, strongest effect — recovered as a dose-response rather than at a single point. That monotonicity is what licenses the move: the phenomenon was *present and under-driven*, not absent, so escalating step size revealed it rather than manufacturing it. All Setting 1 experiments use $\text{lr} = 0.1$. We record the substitution rather than presenting the learning rate as the natural knob, because the effective step size that results is high and a reader should be able to discount for it (Section 8).

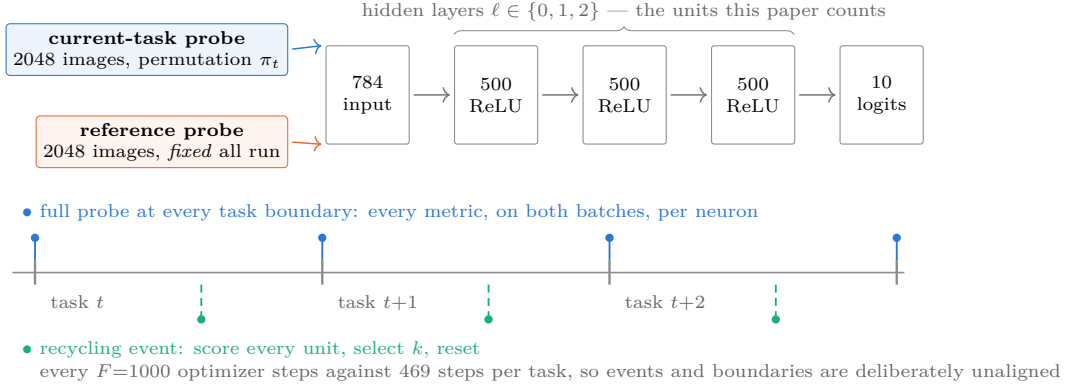


Figure 1: **Setting and instrument.** A 784–500–500–500–10 ReLU MLP on online permuted MNIST: each task is a fresh input permutation seen for a single pass, and the outcome measure is online accuracy, scored on the forward pass taken *before* each update. Every death metric is evaluated on two probe batches — the current task’s distribution and a reference distribution fixed for the entire run — and both are logged, because the gap between them is what separates a unit that is dead from one that is merely silent on the permutation being trained on right now. Per-neuron state is written at every task boundary; recycling events are governed by an optimizer-step counter rather than by task index, so they fall inside tasks rather than at their edges.

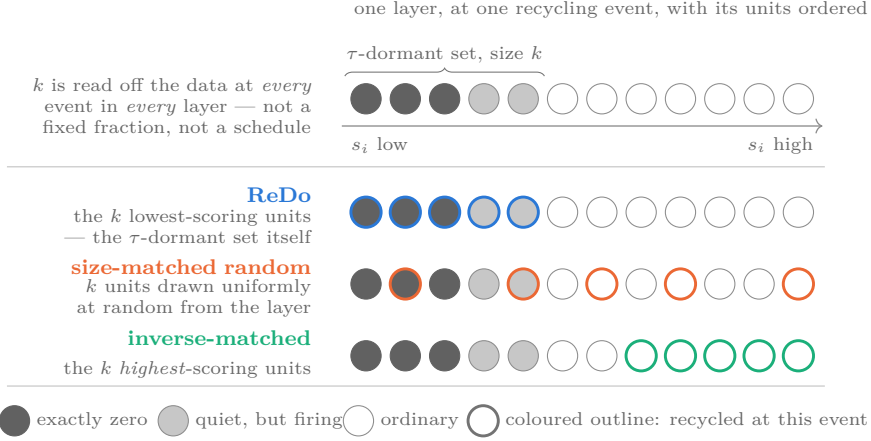


Figure 2: **The three recycling arms.** At every event, in every layer, the τ -dormancy score is computed and k is set to the size of the τ -dormant set. All three arms then recycle exactly k units by the identical procedure — re-initialise the incoming weights from the layer’s original initialisation distribution, zero the outgoing weights (Sokar et al., 2023) — and differ only in *which* k . Because k is recomputed per layer per event rather than fixed as a fraction or a schedule, the comparison isolates the effect of targeting from the effect of dose. Circles are units; their shading is the state measured on the probe batch, which is independent of the score that selects them.

Two further settings. *Activation sweep:* ReLU, LeakyReLU (Maas et al., 2013), GELU (Hendrycks & Gimpel, 2016), SiLU (Elfwing et al., 2018) and tanh on the same task stream, one configuration field. *Convolutional channels:* a small CNN on label-shuffled CIFAR-10, where the unit is a channel and every metric must reduce over (N, H, W) rather than N . The CIFAR arm’s learning-rate calibration failed — see Section 8 — so only its definitional measurement is reported.

4.2 The three recycling arms

ReDo is implemented exactly as specified in Sokar et al. (2023)’s Algorithm 1: every $F = 1000$ optimizer steps, compute s_i^ℓ on a 64-example batch (their default), then for every unit with $s_i^\ell \leq \tau$ re-initialise the incoming weights by sampling from that layer’s *original* initialisation distribution and set the outgoing weights to *zero*. Zeroing the outgoing weights is what preserves the function the network currently computes while restoring gradient flow; omitting it turns ReDo into a far more destructive intervention that still produces plausible-looking curves. A unit test asserts that at $\tau = 0$ the network’s outputs are unchanged to numerical tolerance across a recycling event.

The two controls, and the reason for their design, are the experimental core of this paper (Figure 2):

Size-matched random(τ). At every event, in every layer, compute the τ -dormant set, set k to its cardinality, and then recycle k units drawn *uniformly at random* from that layer by the identical procedure. *k is recomputed at every event and in every layer.* It is not a fixed fraction and not a schedule.

Inverse-matched(τ). The same k , but the k *highest*-scoring units — the ones least likely to be dead.

The distinction from Sokar et al.’s own random baseline, which resets a fixed percentage of units on a cosine schedule, is the entire point. A fixed-percentage baseline varies *how many* units are recycled at the same time as *which* ones, so a gap between ReDo and that baseline is attributable to either. Matching cardinality per layer per event removes the first source of variation, leaving only the second. As it turns out (Section 5.1) the realised dose still differs between arms, but in the direction that works *against* the convenient conclusion, which we report in full.

4.3 Pre-registration, statistics, and what was frozen

What was frozen, and when. The analysis plan — outcome measure, task windows, decision thresholds, seed counts, and the reproduction-gate criterion — was committed as `configs/analysis_plan.json` at 2026-08-05T14:09:03Z, before any experimental data existed, and was never edited. The early window is tasks 1–10; the late window is tasks 151–200; all headline numbers are late-window.

Decision thresholds. Two, both pre-registered. A difference is called *no difference* ($\approx$) when its 95% CI contains zero *and* $|\Delta| < 1pp$; it is called *much greater* ($\gg$) when $\Delta > 2pp$ *and* the CI excludes zero. Anything else is *inconclusive*, and the pre-specified response to inconclusive is to add seeds, never to adjust a threshold. We report the rule’s verdict and the effect size together throughout, including where the rule’s verdict is unhelpful (Section 5.1).

Statistics. The interquartile mean with stratified bootstrap 95% confidence intervals, 10,000 replicates, resampling seeds independently within each task column (Agarwal et al., 2021). Never a bare mean over seeds. Arm comparisons are *paired*: seed s of every arm starts from the identical initialisation and sees the identical task stream, so the bootstrap resamples the same seed indices from both arms, which removes the initialisation variance that would otherwise swamp a one-point effect.

Calibration is separated from pre-registration. Learning rate, batch size, width, depth, number of tasks and dataset are *calibrated*, and every change to them is recorded with a date and a reason in a deviations log before the run. Calibrating a setting so that the phenomenon under study is present at all is a manipulation check; the published work being replicated calibrated its own setting the same way. What must not move, and did not, are the outcome measure and the thresholds by which it is judged.

Determinism. Seeded `torch`, `numpy` and `random`; `cudnn.deterministic`; weight resampling performed on CPU so that a recycling event is identical across hardware. This is demonstrated rather than asserted: on three separate occasions a set of configurations was accidentally re-executed in a fresh session, twice on different hardware and once on a different account, and every run reproduced *bit-identically* — every online-accuracy value equal at `atol=0`.

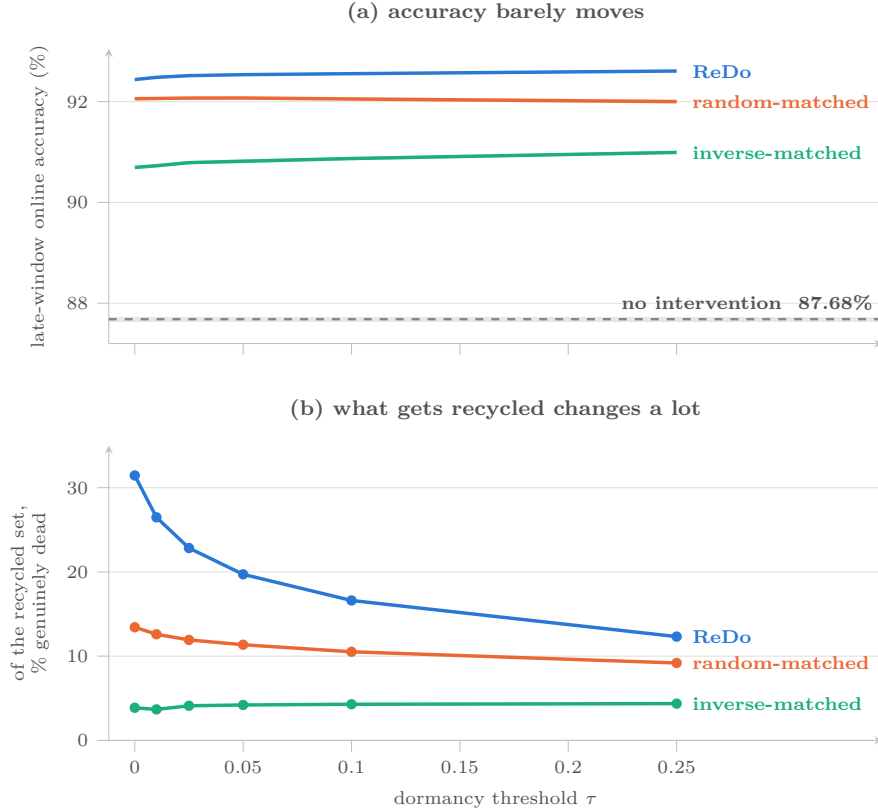


Figure 3: **The dormancy threshold changes what gets recycled far more than it changes the result.** *Upper:* late-window online accuracy against τ for the three arms, with the no-intervention baseline as a reference band. *Lower:* the pre-registered headline quantity — of the units actually recycled, the fraction that were exactly zero on the probe batch, pooled over layers and events. Over the full τ range ReDo’s target set goes from 31.4% dead to 12.3% dead, a factor of 2.5, while its accuracy moves 0.169 *pp* [0.143, 0.196] — real, but 3.6% of the +4.76 *pp* that recycling buys over doing nothing. Each point is 10 seeds; bands are stratified bootstrap 95% CIs. Composition is shown on the current-task probe batch, as the frozen plan specifies; on the fixed reference batch the same quantity runs 27.6% $\rightarrow$ 13.5%.

5 Results

One subsection per claim, in the order they were pre-registered rather than the order that flatters them: the recycling comparison (Section 5.1), the definitional disagreement (Section 5.2), the two regularisation anomalies (Section 5.3), the survival analysis (Section 5.4), and the optimizer arm (Section 5.5), which contains a pre-registered hypothesis that did not confirm and is reported as such. Section 5.6 collects all of them against the thresholds they were judged by. Throughout, “late window” means tasks 151–200, every interval is a stratified bootstrap 95% CI over seeds, and every arm comparison is paired on seed.

5.1 C1: ReDo’s benefit does not require targeting dead units

Recycling works, which is worth establishing first. Whether ReDo helps at all in continual supervised learning had not been tested. It does: +4.76 to +4.92 *pp* over the no-intervention baseline at every τ , every CI excluding zero, all far past the pre-registered $\gg$ threshold (Table 2). It also strongly suppresses dead units — DEAD-EXACT falls from 20.8% to 1.5–2.2% and effective rank rises from 100 to 165–192. Stated on their own, those two numbers are exactly consistent with “ReDo works by preventing death”, and we state them plainly rather than burying them, because the rest of this section is the argument that they are not the mechanism.

Table 2: The τ -sweep: 190 runs, 4 arms, 6 thresholds, 10 seeds, $lr = 0.1$. Late-window online accuracy; the no-intervention baseline is 87.683% [87.634, 87.730]. “dead share” is the pre-registered headline quantity, the exactly-zero fraction of the set actually recycled. “ k ” is the mean number of units recycled per layer per event — the realised dose. Differences are paired IQM with stratified bootstrap 95% CIs. Every *vs. none* entry clears the pre-registered $\gg$ bar.

τ	ReDo			Size-matched random		
	acc. (%)	Δ vs. none	dead share	acc. (%)	Δ vs. none	dead share
0	92.438	+4.755	31.4%	92.058	+4.375	13.4%
0.01	92.482	+4.799	26.5%	92.062	+4.379	12.6%
0.025	92.516	+4.832	22.8%	92.071	+4.387	11.9%
0.05	92.535	+4.852	19.7%	92.072	+4.389	11.4%
0.1	92.555	+4.872	16.6%	92.053	+4.370	10.5%
0.25	92.607	+4.924	12.3%	92.001	+4.318	9.2%

τ	Inverse-matched			ReDo – random, paired		
	acc. (%)	Δ vs. none	dead share	Δ (pp)	95% CI	frozen verdict
0	90.694	+3.011	3.9%	+0.380	[0.346, 0.415]	inconclusive
0.01	90.728	+3.045	3.7%	+0.419	[0.384, 0.454]	inconclusive
0.025	90.790	+3.106	4.1%	+0.445	[0.405, 0.483]	inconclusive
0.05	90.817	+3.134	4.2%	+0.463	[0.426, 0.501]	inconclusive
0.1	90.871	+3.188	4.3%	+0.502	[0.463, 0.540]	inconclusive
0.25	90.992	+3.309	4.4%	+0.606	[0.568, 0.647]	inconclusive

The pre-registered C1 pattern fires. The plan specified in advance that if accuracy improves with τ while the truly-dead fraction of the recycled set falls, C1 is confirmed. That is what happens, and Figure 3 is the whole comparison in one place: dead share 31.4% $\rightarrow$ 12.3%, accuracy 92.438 $\rightarrow$ 92.607. But the honest reading is stronger than the pattern and weaker than the slope. Across the entire τ range ReDo gains +0.169 *pp* [0.143, 0.196] — 3.6% of the +4.76 *pp* it gains by being switched on at all. Widen the threshold by a factor of 25, halve the fraction of the target set that is genuinely dead, and almost nothing happens to the result.

The control: random selection recovers 88–92% of the benefit. Size-matched random recycling gains +4.32 to +4.39 *pp* over baseline against ReDo’s +4.76 to +4.92 *pp*. It recovers 92.0% of ReDo’s benefit at $\tau = 0$ and 87.7% at $\tau = 0.25$, while the sets it recycles are 86.6–90.8% alive and its mean selected Sokar score is 1.00 at every τ — exactly the layer average, which is what uniform selection must produce and which we use as an instrument check.

We do not claim this gap is zero. ReDo beats size-matched random by 0.380 to 0.606 *pp* with every CI excluding zero. Targeting contributes something real. It contributes roughly a tenth of the total effect; the other nine tenths are obtained by recycling units chosen at random, nine in ten of which were alive.

The frozen rule returns *inconclusive*, and we report that. Neither pre-registered branch fires for ReDo versus random: $|\Delta|$ is far below the 2 *pp* bar for $\gg$, but every CI excludes zero so it does not qualify as $\approx$ either. We record, without acting on it, that the pre-specified response — add seeds — cannot change this verdict: the CIs are already ± 0.04 *pp* and more seeds tighten them, while $\approx$ requires a CI containing zero, which no amount of power produces for a true nonzero effect. That is a limitation of the decision rule, not a result, and we report the rule’s verdict beside the effect size rather than amending a plan that was frozen for exactly this reason.

Inverse-matched: the pre-registered sanity check failed, and the failure is the strongest evidence in the section. The plan predicted that recycling the k *highest*-scoring units would collapse performance, replicating Sokar et al.’s Figure 15. It does not collapse. Recycling the units least likely to be dead — mean selected score ≈ 2.0 , only 3.9% of them exactly zero — still beats the baseline by +3.01 to

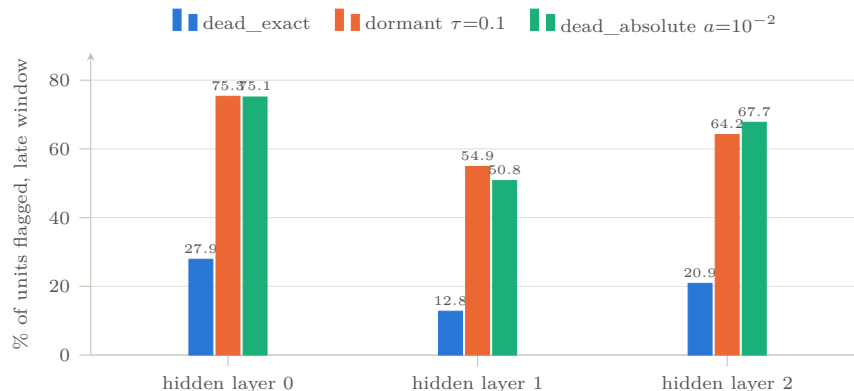


Figure 4: **Three published definitions, one set of activations, per layer.** All quantities are computed on the same forward pass over the same 2048-example probe batch, so the spread between them is definitional, not statistical.

+3.31 *pp*, every CI excluding zero, recovering 63–67% of ReDo’s benefit. The arm designed to be maximally destructive captures two thirds of the effect. We report this as a domain difference from their RL setting rather than as a bug: the composition table confirms the arm recycled the units it was meant to, and the same code path produced the other two arms.

Ordering the three arms by how dead their target sets are gives a monotone relationship with benefit — ReDo (12–31% dead, +4.8 *pp*), random (9–13%, +4.4 *pp*), inverse ($\approx 4\%$, +3.2 *pp*). So targeting is not nothing. But the whole span from “almost entirely alive” to “as dead as the method can find” is worth 1.7 *pp* against a 4.8 *pp* effect.

The dose confound runs the wrong way, twice. The obvious alternative explanation is that ReDo simply perturbs more. It does not. Because k is recomputed from each run’s own state, and because the arms’ networks diverge, the realised dose differs: size-matched random recycles 131 units per layer per event at $\tau = 0$ against ReDo’s 101 (+30%), and inverse-matched recycles 247 (2.4 $\times$). Both controls received a *larger* perturbation and did *worse*. Whatever ReDo’s small advantage is, it is not dose.

This is a design consequence worth naming: matching k within a run to that run’s own dormant set is what the pre-registration specified, and once the arms diverge their dormant counts diverge too. A *yoked* control — replaying ReDo’s per-event k into the random arm — would remove even this, and is the cleaner design for future work. It would tighten the comparison in the direction that already favours our conclusion.

Robustness. The composition is stable across training rather than an early-training artefact: at $\tau = 0$ the dead share of ReDo’s recycled set is 28.3% over tasks 1–25 and 32.3% over tasks 126–200. And even at $\tau = 0$ — Sokar et al.’s “only provably dead units” setting — ReDo recycles 20.1% of the entire network, of which 68.6% is alive. That gap is not ours: it arises because the selection score is computed on their default 64-example batch while DEAD-EXACT is measured on 2048. The 64-example default is doing a great deal of unexamined work.

5.2 C2: the definitions do not measure one thing

Six measurements in this subsection make the same point from different directions, and they are listed here so the claim can be counted rather than taken on trust: (i) on one set of activations the definitions span a factor of 3.5; (ii) under tanh three of the four read exactly zero on the network with the largest plasticity loss in the study; (iii) under LeakyReLU, GELU and SiLU the *Nature* paper’s definition reads exactly zero while the others keep flagging units, and for GELU whether it does depends on floating-point precision; (iv) in convolutional layers that same definition finds essentially no dead channels while most spatial positions are silent; (v) the count changes materially with which distribution does the probing; and (vi) the survival

Table 3: **Change the activation function and the definitions stop agreeing.** Late window, 25 + 15 runs. The tanh row is at its own calibrated learning rate; at $\text{lr} = 0.1$ tanh diverges to chance and is excluded from every table and figure. “—” marks SATURATED as undefined for unbounded activations, which is what the code returns; reporting 0 there would be a category error. Drop is early minus late window.

activation	lr	late acc. (%)	drop (pp)	DEAD-EXACT	DORMANT $_{\tau}$ [0.1]	DEAD-ABS $_a$ [10 $^{-2}$]	SATURATED
ReLU	0.1	87.71	4.51	20.67%	64.90%	64.71%	—
LeakyReLU	0.1	90.05	2.31	0.00%	4.62%	0.00%	—
GELU	0.1	88.44	4.82	0.00%	19.08%	20.49%	—
SiLU	0.1	89.70	3.22	0.00%	1.14%	1.54%	—
tanh	0.003	87.65	0.48	0.00%	0.00%	0.00%	0.00%
tanh	0.01	88.42	2.71	0.00%	0.00%	0.00%	1.24%
tanh	0.03	79.11	13.00	0.00%	0.00%	0.00%	13.84%

analysis of Section 5.4 shows the largest disagreement of all — a revival rate of 60% or 22% depending on the probe.

(i) **A factor of 3.5 on identical activations.** At the operative setting ($\text{lr} = 0.1$, late window, no intervention), the same activations yield: DEAD-EXACT 20.7%; DEAD-ABS $_a$ at $a = 10^{-6}$ 20.7%, at 10^{-4} 25.7%, at 10^{-2} 64.7%; DORMANT $_{\tau}$ at $\tau = 0.1$ — ReDo’s operating point — 64.9%, at $\tau = 0.25$ 72.6%. A reader told “21% of this network is dead” and a reader told “65% of this network is dormant” are being told about the same network in the same state.

It is not an artefact of a collapsed layer. A wholly dead layer would make s_i^{ℓ} degenerate (0/0, defined as 0), flagging every unit in it as dormant at every τ and inflating the pooled figure. That is not what is happening. The per-layer DORMANT $_{\tau}$ [0.1] breakdown is 75.3%/54.9%/64.2% across the three hidden layers, with DEAD-EXACT at 27.9%/12.8%/20.9%; no layer is close to wholly dead (Figure 4). The full decomposition — every definition at every threshold, per layer — is Table 8, and the spread between definitions is present *within* each layer rather than being produced by pooling across them.

(ii) **The tanh result.** At its calibrated learning rate (0.03, chosen by the same dose-response ladder used for the main setting; Table 3), a tanh network loses 13.00 *pp* of online accuracy from the early to the late window — the largest plasticity loss of any configuration in this study, larger than any ReLU arm — while ending at 79.11%, nowhere near the 10% chance level that would indicate mere divergence. And DEAD-EXACT, DORMANT $_{\tau}$ at every τ , and DEAD-ABS $_a$ at every a all report **exactly** 0.00%. Only SATURATED sees anything at all (13.84% on the current-task probe, 25.87% on the reference probe), and SATURATED is undefined for every activation function used by the four papers this work engages.

A network can therefore lose more plasticity than any ReLU configuration we measure and contain, by all three of the field’s published definitions, not one dead unit. This is the single strongest demonstration in the paper that the metrics are not tracking one underlying quantity. It also means the reproduction gate’s dead-unit criterion is unsatisfiable for tanh at any learning rate — DEAD-EXACT is identically zero, so its rank correlation with task index is zero by construction — which is the dissociation case the pre-registered plan says to stop and report rather than work around.

(iii) **Smooth activations: weaker than “by construction”, and dtype-dependent.** Under GELU and SiLU, DEAD-EXACT is 0.00% while DORMANT $_{\tau}$ [0.1] flags 19.1% and 1.1% respectively and DEAD-ABS $_a$ [10 $^{-2}$] flags 20.5% and 1.5%. GELU has the *largest* plasticity drop of the four unbounded activations (4.82 *pp*) and, by the *Nature* paper’s definition, no dead units at all.

The mechanism deserves care, because the intuitive claim is wrong in an instructive way. For SiLU the vanishing of DEAD-EXACT genuinely is by construction: $\sigma(x)$ requires $x \approx -90$ before it underflows, which nothing reaches. For GELU it is not. $\text{GELU}(x) = x\Phi(x)$, and in float32 $1 + \text{erf}(x/\sqrt{2})$ rounds to zero at $x \leq -5.55$, a pre-activation trained networks reach routinely — so GELU units *can* register as exactly dead,

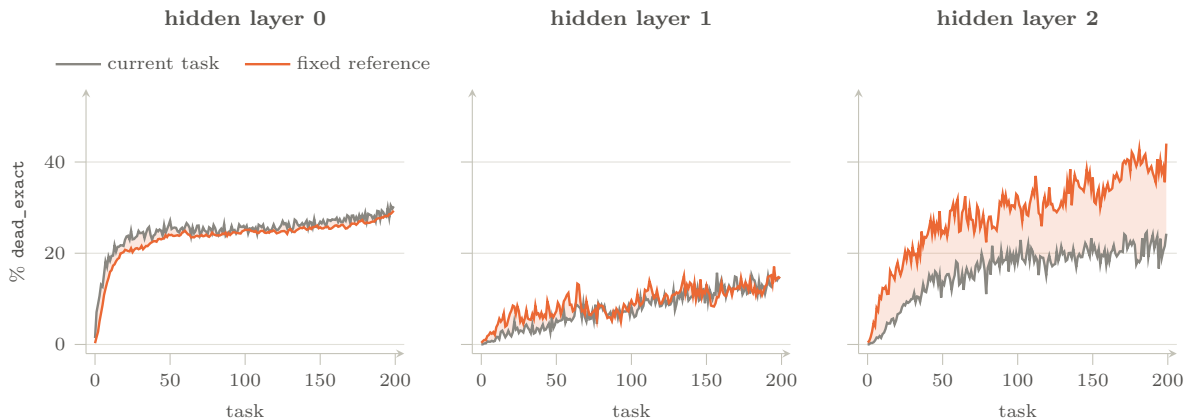


Figure 5: **dead-exact measured on the current task’s distribution versus a fixed reference distribution, per layer.** The two agree in layers 0 and 1 and diverge sharply in the last hidden layer. Distribution-relative death is a last-layer phenomenon, which is a sharper claim than the pooled figure.

through a floating-point underflow. The threshold is -5.55 in float32 and -8.40 in float64: the same unit, same weights, same inputs, is dead in one precision and alive in the other. Our trained GELU networks did not reach it, so the measured value is 0.00%, but the reason is empirical rather than definitional. A death metric whose value depends on floating-point precision is not measuring a property of the network.

(iv) Convolutional channels. In a convolutional layer the unit is a channel and its activation is a feature map, so “dead” must mean zero across all inputs *and* all spatial positions. Under that generalisation, in the three convolutional layers of a small CNN on label-shuffled CIFAR-10, DEAD-EXACT finds 0.00%, 0.91% and 0.00% of channels dead while 81–88% of all spatial positions in those layers are silent, and $\text{DORMANT}_\tau[0.1]$ flags 21–35%. The fully-connected layer in the same network reports 26.3% dead. The definition is therefore not merely activation-dependent but layer-type-dependent: the third convolutional layer is 86% spatially silent with not one of its channels counted as dead. *This measurement is reported on its own*; the CIFAR arm’s learning-rate calibration failed (Section 8), so no plasticity or intervention result from that setting is used anywhere in this paper. A statement about what a definition counts does not require the network to be losing plasticity.

(v) Death is partly distribution-relative. Comparing the two probe batches at $\text{lr} = 0.1$: pooled DEAD-EXACT is 20.7% on the current task’s distribution and 26.0% on the fixed reference. Units stay alive for the permutation currently being trained on and fall silent on a distribution the network has not seen recently. Per layer the effect localises completely: 27.9% vs. 26.9% (layer 0) and 12.8% vs. 12.1% (layer 1) show no gap, while layer 2 reads 20.9% vs. 37.2% (Figure 5). None of the four source papers can observe this, because none of them probes a second, fixed distribution. Section 5.4 shows the same asymmetry dominating a much larger quantity.

5.3 C3: helping plasticity while killing more neurons

Both target observations pre-registered for this experiment fire (Table 4, Figure 6).

L_2 improves accuracy while nearly doubling dead units. At $\lambda = 10^{-4}$, L_2 regularisation gains $+1.50 pp$ [1.41, 1.59] over plain backpropagation while DEAD-EXACT rises from 20.67% to 40.17% and effective rank falls from 100.2 to 50.1. Higher accuracy, nearly twice the dead units, half the effective rank. The same holds on the reference probe (26.02% $\rightarrow$ 49.36%), so the anomaly is not an artefact of which distribution is used to probe.

The dose–response sharpens the point rather than softening it. Accuracy is an inverted U in λ : 10^{-4} helps, 10^{-3} costs 2.38 pp , 10^{-2} costs 11.86 pp . Dead units are *not* monotone either — they peak at $\lambda = 10^{-3}$

Table 4: **The two anomalies, replicated.** 35 runs, 5 seeds, lr = 0.1, late window. Δ is paired against plain backpropagation. DEAD-EXACT is given on both probe batches because the C3 claim should not depend on that choice, and does not.

arm	acc. (%)	Δ (pp)	95% CI	DEAD-EXACT cur.	DEAD-EXACT ref.	erank
plain backprop	87.71	—	—	20.67%	26.02%	100.2
L_2 , $\lambda=10^{-4}$	89.22	+1.50	[1.41, 1.59]	40.17%	49.36%	50.1
L_2 , $\lambda=10^{-3}$	85.34	-2.38	[-2.65, -2.11]	66.52%	72.47%	14.5
L_2 , $\lambda=10^{-2}$	75.85	-11.86	[-12.70, -11.04]	60.38%	66.20%	7.1
shrink & perturb	89.66	+1.94	[1.87, 2.02]	7.18%	6.76%	46.3
dropout 0.1	83.31	-4.40	[-4.51, -4.30]	22.45%	31.90%	109.9
online normalisation	90.87	+3.16	[3.08, 3.23]	28.61%	39.73%	146.4

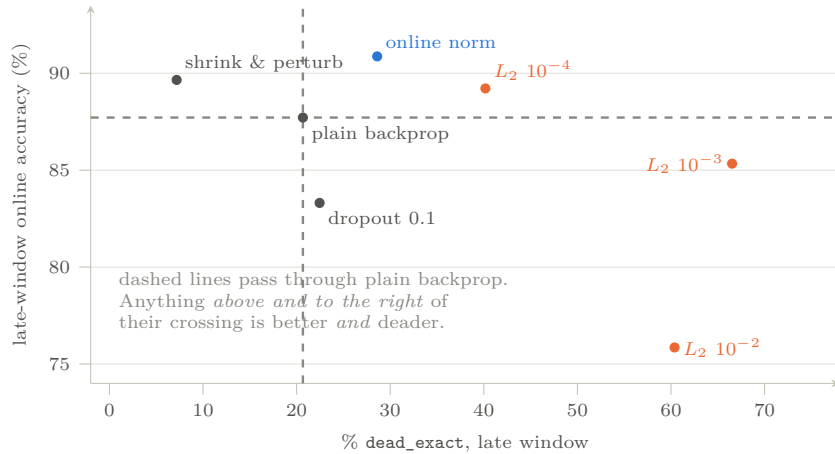


Figure 6: **The C3 dissociation is two-dimensional.** Accuracy against dead units, one point per arm. If dead units were the pathology, the arms would lie on a downward line. L_2 at its beneficial dose and online normalisation are both up and to the right — better *and* deadier.

(66.5%) and fall back at 10^{-2} (60.4%), by which point effective rank has collapsed to 7.1 and the network is simply small. So the anomaly is specifically that *within the beneficial dose* the two quantities move in opposite directions. That is a stronger and more awkward statement than “ L_2 increases death”.

Online normalisation increases the pathology it was designed to prevent. Online Normalization (Chiley et al., 2019) is the best-performing arm we measure, at +3.16 pp [3.08, 3.23] over backpropagation — clearing the pre-registered $\gg$ threshold, the only C3 arm to do so — while carrying 28.61% dead units against backpropagation’s 20.67%, a 38% increase (39.73% vs. 26.02%, a 53% increase, on the reference probe). It also has the highest effective rank of any arm (146.4). The method works; its stated mechanism is not why.

The counterexample that keeps this honest. Shrink-and-perturb (Ash & Adams, 2020) gains +1.94 pp *and* reduces DEAD-EXACT to 7.18%. Not every helpful method increases death. This is precisely why the claim must be stated as a dissociation — dead-unit count and plasticity can move independently, in either relative direction — rather than as a law that death is good. Dropout (Srivastava et al., 2014) is the fourth quadrant: it hurts accuracy (−4.40 pp) while barely changing death (22.45%).

Mechanism. The pattern across arms is consistent with dead-unit count tracking *parameter-norm and scale* effects rather than lost capacity. L_2 and online normalisation both act on activation and weight scale; both shift many units below the exactly-zero threshold; and both improve trainability by keeping the effective step size in a useful range. Shrink-and-perturb, which periodically rescales *and* re-randomises, moves scale

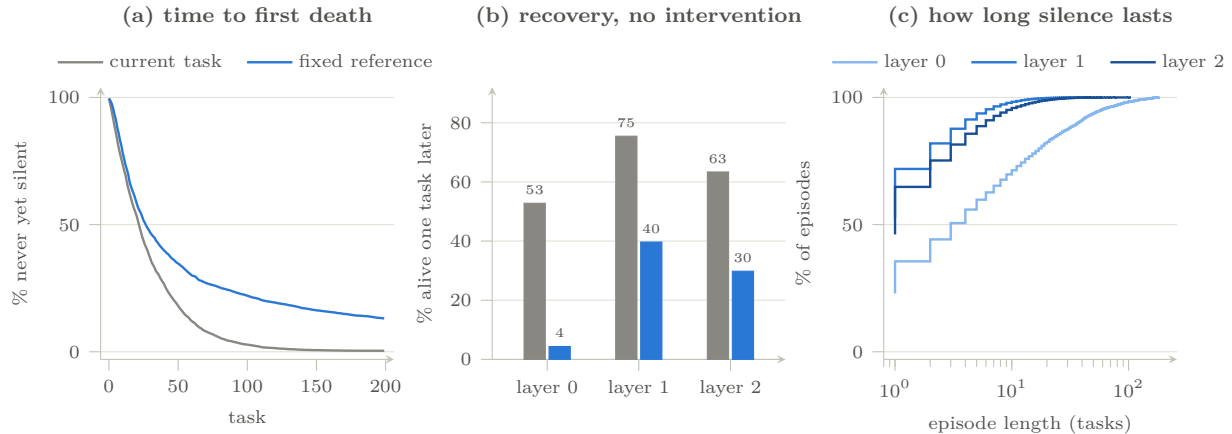


Figure 7: **Neuron mortality as a survival process.** Per-neuron state logged at every one of 200 task boundaries, the five $\text{lr} = 0.1$ runs, with no intervention of any kind. (a) time to first death; (b) probability that a unit dead at one boundary is alive at the next, per layer — same two series and same colours as (a), so the legend above (a) serves both panels; (c) how long silence lasts, as an empirical CDF over the 68,440 closed episodes on the reference probe.

without pushing units against the ReLU hinge, and its death count falls. Under this reading a DEAD-EXACT count is partly a readout of where a layer’s activation distribution sits relative to zero, which is a scale property, and only partly a readout of lost capacity — which is what Section 5.4 independently supports.

5.4 C4: death is reversible

We logged the full state of every unit — mean absolute activation, exact-zero flag on both probes, incoming and outgoing weight norms, per-neuron gradient norm — at every one of the 200 task boundaries, and treat each unit as a subject in a survival study. The runs analysed here carry *no intervention*, so nothing below can be an artefact of recycling. Figure 7 gives the three views the rest of this subsection quantifies: when units first die, whether they come back, and how long they stay silent.

A fifth of dead units come back on their own. On the fixed reference distribution, a unit that is DEAD-EXACT at one task boundary is alive again at the next with probability 21.71% [21.37, 22.21], pooled across layers, with no intervention applied.

layer	$P(\text{dead} \rightarrow \text{alive}), \text{one task later}$	95% CI
0	4.36%	[4.22, 4.47]
1	39.68%	[38.27, 42.93]
2	29.82%	[29.51, 31.71]
pooled	21.71%	[21.37, 22.21]

Death is a state units churn through, not a fate they arrive at. 87.3% of all units die at least once, against a late-window prevalence of only 26%. Of the 68,440 death episodes, the median closed episode lasts 2 tasks, 46.2% last exactly one task, and only 3.2% are still open at task 199.

The first layer is the exception, and should be reported as one. Layer 0’s recovery rate is 4.4% against 30–40% in layers 1 and 2; its median closed episode is 4 tasks against 1 and 2; and 12.6% of its episodes never end, against 2.0% and 2.5%. Permanent death is real, but in this setting it is a first-layer phenomenon. That is consistent with the mechanism: layer 0 sees the raw permuted input, so its pre-activations are not rescued by upstream parameters changing.

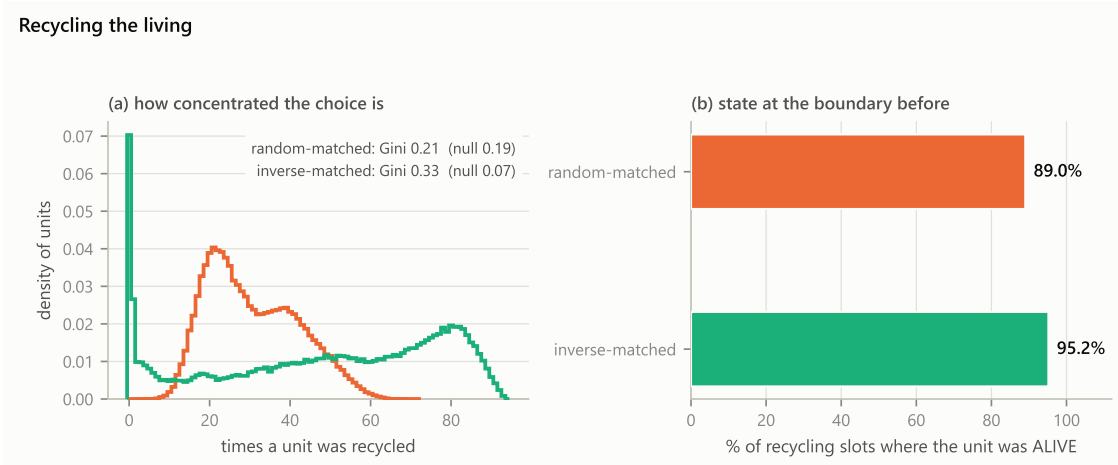


Figure 8: **Which units recycling keeps choosing, and what state they were in.** Per-unit recycle counts against a null that holds k fixed per task and per layer and redraws only *which* units, so any excess concentration is targeting rather than dose.

Table 5: **Optimizers.** 20 runs, 5 seeds, late window. SGD runs at its own calibrated learning rate per the protocol, so the SGD-vs-Adam accuracy comparison is not like-for-like; the Adam-default-vs-Lyle-tuned contrast, at identical learning rate, is clean.

arm	lr	acc. (%)	drop (pp)	DEAD-EXACT	erank
SGD + momentum	0.1	87.71	4.51	20.67%	100.2
Adam, defaults ($\epsilon=10^{-8}$, $\beta_2=0.999$)	10^{-3}	90.28	3.19	59.30%	50.5
Adam, tuned ($\epsilon=10^{-3}$, $\beta_2=0.9$)	10^{-3}	90.46	2.25	24.60%	120.8
AdamW	10^{-3}	91.18	2.26	58.37%	54.9

Do not quote the current-task version of this number. On the current task’s probe batch the same quantity reads 59.91% [59.72, 60.22]. That is not a better estimate of revival; it is mostly the input distribution moving. The permutation changes at every boundary, so a unit silent on task t ’s inputs may fire on task $t+1$ ’s without any parameter having changed. The reference batch is fixed and cannot produce that artefact, which is why it carries the claim. The gap between 60% and 22% is itself a C2 result: most of what naively looks like spontaneous recovery is distribution shift.

Why this matters for the rest of the paper. Every method this paper engages is premised on dead units being lost capacity that has to be actively restored. If a fifth of them restore themselves within one task, the premise is weakened before any intervention is applied — and a recycling method that appears to “revive” units is competing against a process that revives them anyway.

Recycling selects the living. Across every recycling event in the sweep, the fraction of the selected set that was alive is 86.6–90.8% for size-matched random and 95.6–96.3% for inverse-matched. The second number is that arm behaving exactly as designed and is not evidence on its own; its value is that the two arms sit at opposite ends of the targeting scale, which calibrates the measure. The concentration analysis (Figure 8) confirms this: on the random arm, per-unit recycle counts match a same-dose null to three decimals (enrichment 1.001/1.005/0.999 across layers, Gini 0.139–0.221 against a null of 0.138–0.222), which is what a negative control must produce; on the inverse arm the same units are chosen repeatedly, roughly $5\times$ more concentrated than dose alone predicts.

5.5 C5: Adam-induced death is real; its predicted timing is not

The effect is real and large. Table 5 gives the four arms. Adam (Kingma & Ba, 2015) at its default hyperparameters carries 59.30% dead units against SGD’s 20.67%, with effective rank halved (50.5 vs. 100.2). The two-hyperparameter change recommended by Lyle et al. (2023) — raising ϵ to 10^{-3} and lowering β_2 to 0.9 — cuts this to 24.60% at an identical learning rate, a $2.4\times$ reduction, and restores effective rank to 120.8. AdamW’s decoupled weight decay (Loshchilov & Hutter, 2019) does not rescue it (58.37%).

And it dissociates from plasticity, again. Adam carries roughly three times SGD’s dead units *and* a smaller plasticity drop (3.19 vs. 4.51 *pp*). This is the fourth independent dissociation in the paper. The accuracy comparison across optimizers is not like-for-like, since the learning rates differ by design; the dead-unit comparison is, since it is a property of the activations either way.

The pre-registered timing hypothesis did not confirm. We report it as a null. The plan predicted a death spike in the first steps after each task switch, when Adam’s first-moment estimate $\hat{m}$ has updated aggressively but the second-moment estimate $\hat{v}$ has not — the mechanism of Lyle et al. (2023). To test it we added intra-task probing at 25-step resolution, yielding 228,000 probe rows across the C5 runs. Pooling over tasks 50–199 by steps-since-switch:

arm	step 0	25	50	100	200	450
SGD	22.94	25.38	22.95	21.17	19.82	18.40
Adam, defaults	63.18	57.42	56.21	55.98	55.94	55.86
Adam, tuned	26.91	23.49	23.02	22.87	22.06	20.54
AdamW	63.57	57.12	56.08	55.76	55.58	55.38

SGD shows the predicted spike (+2.44 *pp* at step 25). Adam does not: it declines monotonically from the switch onward, which is the reverse of the hypothesis. Adam-induced death is real but it **accumulates across tasks** (25.5% over tasks 0–19, 64.9% over tasks 180–199) rather than within them. Within a task, Adam’s layers move in opposite directions: layer 0 *gains* dead units over the course of a task (24.4 $\rightarrow$ 27.6%) while layers 1 and 2 recover (76.3 $\rightarrow$ 64.2% and 88.8 $\rightarrow$ 75.8%).

Two limitations could in principle produce this null, and neither affects the cross-task accumulation result. The probe grid is 25 steps, so a spike confined to the first ten steps would be invisible — SGD’s was caught only because it is broad; a log-spaced grid would settle it. And the reference probe was disabled for these runs to halve probe cost, so the step-0 reading conflates “the unit is dead” with “the permutation just changed”, which is exactly what the reference batch exists to separate. Both should be fixed before this null is treated as final.

5.6 Summary

Table 6 collects every claim against the threshold it was judged by. Four independent dissociations between dead-unit count and plasticity appear across the study: at $\text{lr} = 0.001$ dead units nearly triple while accuracy *improves*; Adam carries three times SGD’s dead units with a *smaller* plasticity drop; GELU has the largest drop of the unbounded activations and *zero* dead units by the standard metric; and L_2 at its beneficial dose doubles dead units while improving accuracy. Alongside those, C4 shows dead units returning on their own.

6 Implications for measurement practice

Each of the following is a change a paper in this area can adopt immediately, and each follows from a specific result above rather than from general caution.

1. **Report a size-matched random control alongside any recycling-based intervention** — k recomputed per layer per event, not a fixed percentage and not a schedule. This is the only design that separates the effect of *which* units are recycled from *how many*. A fixed-percentage

Table 6: **Every claim, its effect size, the pre-registered threshold it was judged against, and the verdict.** Thresholds are from the frozen analysis plan and were not adjusted after seeing data. “Inconclusive” is the plan’s own label for a difference below the $\gg$ bar whose CI excludes zero; where it appears we report it together with the effect size, since the rule cannot return $\approx$ for any true nonzero effect.

	claim	effect	pre-registered rule	verdict
C1	ReDo’s benefit does not come from reviving dead units	+0.169 <i>pp</i> across τ while dead share 31.4% $\rightarrow$ 12.3%; random-matched recovers 88–92%	accuracy up while dead share down $\Rightarrow$ C1 confirmed	supported ; strong form refuted
C1’	ReDo $\gg$ size-matched random	+0.38 to +0.61 <i>pp</i> , CI excludes zero	$\gg$ needs $>2\text{ }pp$; $\approx$ needs $CI \ni 0$	inconclusive
C1’’	inverse-matched collapses (Sokar et al., 2023)	+3.01 to +3.31 <i>pp</i> above baseline	collapse predicted	refuted
C2	the definitions disagree materially	3.5 $\times$ spread; 0.00% dead against 13.0 <i>pp</i> lost under tanh	qualitative target observation	established , six ways
C3	L_2 helps while increasing death; online norm increases what it prevents	+1.50 <i>pp</i> with 20.7 $\rightarrow$ 40.2%; +3.16 <i>pp</i> with 20.7 $\rightarrow$ 28.6%	both target observations stated in advance	confirmed , both halves
C4	mortality has characterisable dynamics	21.71% [21.37, 22.21] revival; 87.3% ever dead	analysis-only, no threshold	death is reversible
C5	Adam-induced death is a distinct mechanism	59.3% vs. 20.7% dead, but no post-switch spike	within-task spike predicted	half : real, timing null

baseline varies both at once, and the conflation is what produces an inflated impression of targeting precision: here, the size-matched control recovers 88–92% of the benefit that the targeted method receives credit for.

2. **Never report a single “percent dead” figure without pairing it with a non-layer-relative measure.** On identical activations, DORMANT $_{\tau}$ [0.1] read 64.9% where DEAD-EXACT read 20.7%; the difference is quiet-but-firing units, and one fixed-absolute or exact-zero number reported alongside makes it visible at a glance instead of requiring a follow-up study. The two measures answer different questions and neither is a substitute for the other.
3. **Report death against a fixed reference distribution, not only the current task’s.** The same units read 20.7% dead on the current task’s inputs and 26.0% on a fixed reference, localised almost entirely in the last hidden layer (20.9% vs. 37.2%); and the spontaneous-revival rate reads 60% on one probe and 22% on the other. A reference probe is the only way to separate genuine death from distribution-relative silence, and it costs one extra forward pass per measurement.
4. **Report the spontaneous revival rate as a baseline before crediting any method with “recovering” capacity.** At a 21.7% per-task baseline with no intervention at all, some portion of any reset method’s apparent restoration is units that would have come back regardless. This baseline is cheap: it requires only that per-neuron state be logged at task boundaries, which is a write, not a computation.
5. **When evaluating on anything other than ReLU, do not rely on exact-zero counts.** Under SiLU the count is zero by construction; under GELU it is zero in practice and, worse, depends on floating-point precision; under tanh it is zero while the network saturates. The tanh result here is the sharpest case on record: 13.0 *pp* of plasticity lost with three published definitions all reading exactly 0.00%. Report a saturation criterion for bounded activations and an absolute-magnitude criterion for smooth ones, and report exact-zero as *undefined* rather than 0 where it does not apply.

7 Discussion

The thread. The field measures a proxy that is sometimes present without the pathology and sometimes absent despite it. Present without it: L_2 at its beneficial dose, online normalisation, and Adam all carry more dead units than the baselines they beat. Absent despite it: GELU has the largest plasticity drop of the unbounded activations with DEAD-EXACT at zero, and tanh loses 13.0 *pp* with three definitions at zero. A quantity that behaves this way is not a measurement of lost capacity. It is closer to a readout of where a layer’s activation distribution currently sits relative to the origin — a scale property, which is why interventions that act on scale (L_2 , normalisation, Adam’s per-parameter rescaling) move it so reliably in either direction.

What this does and does not say about recycling methods. They work. ReDo gains +4.8 *pp* in our setting, which had not previously been shown for continual supervised learning at all, and it does suppress dead units and lift effective rank substantially. Nothing here argues for abandoning them. What the evidence does not support is the stated mechanism. Recycling units chosen uniformly at random, roughly nine in ten of which are alive, recovers nearly all of the benefit; recycling the units *least* likely to be dead recovers two thirds. The natural reading is that the operative ingredient is the perturbation — re-randomised incoming weights with zeroed outgoing weights, applied periodically, which resets a subspace of the parameters without changing the function — and that the dormancy score is a convenient way to choose where to apply it rather than the reason it works. That is consistent with Klein et al. (2024)’s survey finding that generic regularisation outperforms dormancy-targeted intervention, and with Lyle et al. (2024)’s decomposition into multiple mechanisms on which single-target interventions are individually insufficient.

Relation to prior negative results. Our findings line up with the three counter-results scattered through the literature rather than contradicting them. Sokar et al.’s observation that pruning dormant units is free is what one expects if those units are not carrying recoverable capacity. Lyle et al.’s plasticity loss without saturated units is the same dissociation we measure four more times. Kooi et al.’s sign reversal under tanh is the phenomenon our tanh arm isolates to the definition itself. Where we diverge is on inverse-matched recycling, which Sokar et al. report as collapsing and we find beats baseline by 3 *pp*. We treat that as a domain difference between supervised continual learning and their RL setting rather than as a correction to their result; the arm ran correctly, and its composition table confirms it recycled the units it was meant to.

Threats to validity. The primary setting is one architecture family on one task stream, and the effective step size (lr = 0.1 with momentum 0.9) is high. The most important guard against reading too much into that is that the central claims are dissociations — pairs of quantities moving in opposite directions — rather than absolute levels, and they appear at five learning rates spanning two decades, four activation functions, four optimizers, two probe distributions, and both a fully-connected and a convolutional architecture. A confound that produced all of those in the same direction would have to be unusually accommodating. The claim most exposed to setting is the magnitude of ReDo’s advantage over random-matched, which is small and could plausibly be a different small number elsewhere; the claim least exposed is that the definitions disagree, which is arithmetic on a single forward pass.

8 Limitations

- **Architecture.** An MLP, plus a small CNN for the channel-level definitional measurement. No large-scale vision or language model.
- **Task streams.** Online permuted MNIST and label-shuffled CIFAR-10. Both are synthetic non-stationarity of a kind the plasticity literature uses routinely, but neither is a natural distribution shift.
- **No transformer arm, and the reason.** We considered and cancelled one. For a gated unit — SwiGLU computes $(W_1x) \odot \sigma(W_2x)$ — “the neuron is dead” is not well defined: gate-death, value-death and product-death are three different events with different gradient consequences, and

no definition covering them exists in the literature. Inventing one is a separate paper, and using a wrong one here would undercut the very argument we are making about definitional care. The three works this paper engages most closely use permuted MNIST, small CNNs and DQN respectively, so this is not a lower bar than the literature being examined — but it is a real open problem and we state it as one.

- **Effective step size.** $\text{lr} = 0.1$ with momentum 0.9 is high. It was reached by the pre-registered failure ladder, and it is a substitute for the correct fix, which was to reduce batch size toward the fully online regime of Dohare et al. (2024) (Section 4). The dose-response is monotone across two decades of step size, so we do not think the phenomenon is an artefact of the endpoint, but a reader should discount accordingly.
- **The CIFAR arm failed calibration and its plasticity results are not reported.** At $\text{lr} = 0.01$ over 50 tasks accuracy *rose* (45.1% $\rightarrow$ 56.6%), so there was no plasticity loss for recycling to address; at $\text{lr} \in \{0.03, 0.1\}$ over 200 tasks the network collapsed entirely by task 30 — loss exactly $\ln 10$, gradient norm zero in every layer, the fully-connected layer 99.8–100% dead. No usable learning rate was found, so only the channel-level definitional measurement from that setting is used. This also exposed a real limitation of our own frozen gate: a totally collapsed network satisfies “accuracy fell by $\geq 3pp$ ” and “dead units rose” *maximally*, so collapse produces the most emphatic possible pass. We report this rather than amending the plan, and added a separate advisory health check (accuracy at chance, loss at $\ln n_{\text{classes}}$, gradient norm zero) that runs alongside the frozen verdict without changing it.
- **The C5 timing hypothesis did not confirm**, with two identified weaknesses in the test itself (a 25-step probe grid, and the reference probe disabled for those runs). Section 5.5.
- **Boundary resolution in the survival analysis.** Per-neuron state is logged at task boundaries, and recycling fires every 1000 steps against 469 steps per task, so a unit that died and revived *within* a task is invisible. The revival rates in Section 5.4 are therefore lower bounds.
- **dead-exact is a knife edge.** A unit whose pre-activation sits just below zero on every reference input crosses back on a small weight change. That is arguably the finding rather than a flaw — it is a property of the field’s definition — but a robustness pass with DEAD-ABS_a at 10^{-4} would settle whether the revived units are predominantly the marginal ones. The logged data retains `mean_abs_act` at float64 precisely so this can be done without re-running anything.
- **The controls are matched within a run, not yoked across arms.** As described in Section 5.1, this leaves a residual dose difference of 30–150% between arms. It runs against our conclusion in every case, so it is conservative here, but a yoked design is cleaner.

9 Conclusion

Four claims, stated as they were in the introduction and now with their evidence attached.

ReDo’s benefit does not require targeting dead units. Across the full τ range the genuinely dead share of the recycled set falls from 31.4% to 12.3% while accuracy moves 0.169 *pp*, 3.6% of the benefit of recycling at all. A control matched in size — not percentage, size, recomputed per layer at every event — recovers 88–92% of the benefit while recycling sets that are 87–91% alive, and recycling the highest-scoring units recovers two thirds. Targeting contributes on the order of a tenth of the effect; the remainder is unrelated to which units are chosen.

The field’s death metrics disagree by a factor of 3.5 on identical activations, systematically rather than noisily, with each definition’s blind spot traceable to a specific design choice — and under tanh three of the four report exactly 0.00% on a network that has lost 13.0 *pp* of plasticity.

Two anomalies reported inside a *Nature* paper replicate and are given a mechanism: L_2 at its beneficial dose improves accuracy while nearly doubling dead units and halving effective rank, and online

normalisation is the best-performing method we measure while carrying 38% more of the units it was designed to prevent. Both are consistent with dead-unit count acting as a readout of activation scale rather than of lost capacity.

A fifth of dead units revive with no intervention at all — 21.71% within one task on a fixed reference distribution, with 87.3% of units dying at least once against a 26% steady-state prevalence — which undercuts the premise, shared by every mitigation method surveyed, that dead units are permanently lost capacity requiring active restoration.

None of this argues against recycling methods, which work. It argues that the quantity the field measures is not the quantity it believes it is measuring, and that the fix is cheap: a size-matched control, a second probe distribution, an absolute measure beside the relative one, and a revival baseline. Those four additions cost one extra forward pass and a column in a log, and any of the results above would have surfaced years earlier had they been standard.

10 Supporting material

The five sections that follow are part of the paper rather than an appendix behind the references, because two of them are load-bearing for how the results should be read. Section 11 reproduces the frozen pre-registration in full: it is what makes “the prediction fired” and “the prediction failed” checkable rather than assertions. Section 12 gives the reproduction gate, which establishes that the setting exhibits plasticity loss before any intervention is applied — a manipulation check, not a finding, which is why it is here and not in Section 5. Section 13 decomposes every death definition per layer, ruling out the one artefact that could manufacture C2. Section 14 holds two supporting figures and Section 15 the compute accounting.

11 The pre-registered analysis plan

The plan was committed as `configs/analysis_plan.json` at **2026-08-05T14:09:03Z**, before any experimental data existed, and was never edited. Its operative contents:

- **Primary outcome:** `online_accuracy` — per task, the fraction of training examples classified correctly on the forward pass taken before the parameter update for that minibatch, averaged over the task’s single pass. Stated in advance: for the dropout arm this is measured under dropout, as it is in Dohare et al. (2024). **Secondary:** eval-mode top-1 accuracy on a fixed 2048-example held-out batch.
- **Windows:** early = tasks 1–10; late = tasks 151–200; trend = tasks 1–200. The pre-training probe is excluded from all windows.
- **Statistics:** IQM (`trim_mean`, 0.25), stratified bootstrap percentile intervals, 10,000 replicates, 95% confidence, seed 0, paired across arms. “Never report bare means over seeds.”
- **Decision thresholds:** $\approx$ requires the 95% CI of the difference to contain zero *and* $|\Delta| < 1.0 pp$; $\gg$ requires $\Delta > 2.0 pp$ *and* the CI to exclude zero; anything else is inconclusive, and the response to inconclusive is “add seeds; never adjust a threshold”.
- **Seeds:** 5 for the gate, C3 and C5; 10 for the τ -sweep.
- **Gate criterion:** mean per-task accuracy over tasks 151–200 must be $\geq 3 pp$ below tasks 1–10 with a stratified-bootstrap 95% CI excluding zero, *and* DEAD-EXACT must rise. The protocol’s phrase “rise monotonically” is not satisfiable as literal per-task monotonicity over 200 noisy measurements; it was operationalised *in the frozen file* as a Spearman rank correlation between task index and the network-pooled DEAD-EXACT fraction, required positive with a CI excluding zero.
- **Failure response, in order:** raise the learning rate; extend to 400 tasks; narrow to width 200. “Weakening the criterion” is listed as forbidden.
- **Stop rule:** if the accuracy criterion passes while the dead-unit criterion fails, stop and report — that dissociation is itself a result.
- **τ -sweep predictions, stated in advance:** (i) accuracy improving with τ while the truly-dead fraction of the recycled set falls $\Rightarrow$ C1 confirmed, ReDo is not a resurrection method; (ii) ReDo $\approx$ random-matched at large $\tau \Rightarrow$ targeting is irrelevant, strongest form of C1; (iii) ReDo $\gg$ random-matched at every $\tau \Rightarrow$ C1 refuted, report it honestly; (iv) inverse-matched collapsing relative to none $\Rightarrow$ sanity check passed. Outcomes: (i) fired; (ii) and (iii) both failed to fire, leaving *inconclusive*; (iv) *failed*.
- **Headline quantity:** “`n_dead_exact` / `k`, pooled over layers and events within a run, measured on the current-task probe batch.” This is what Figure 3’s lower panel plots.

12 The reproduction gate

Before running any intervention we had to establish that the setting exhibits the phenomenon the interventions are meant to address. This is infrastructure validation rather than a finding, which is why it is here rather than in Section 5: putting it in the main results would dilute the four contributions with a manipulation check. Table 7 gives the full learning-rate ladder and Figure 9 plots both of the gate’s criteria against step size.

Table 7: The full learning-rate ladder, 25 runs, 5 seeds each. Drop is early-window minus late-window online accuracy. This is infrastructure validation, not a finding: it establishes that the phenomenon under study is present in the setting before any intervention is applied.

lr	early (%)	late (%)	drop (pp)	95% CI	DEAD-EXACT early → late	verdict
0.1	92.22	87.71	+4.54	[4.45, 4.69]	4.54% → 20.67%	PASS
0.03	92.91	90.24	+2.68	[2.45, 2.84]	0.82% → 14.80%	fail
0.01	91.71	90.22	+1.33	[1.20, 1.37]	0.28% → 7.65%	fail
0.003	88.56	88.44	−0.25	[−0.37, −0.12]	1.04% → 3.71%	fail
0.001	83.92	84.37	−1.18	[−1.28, −1.09]	1.86% → 4.74%	fail

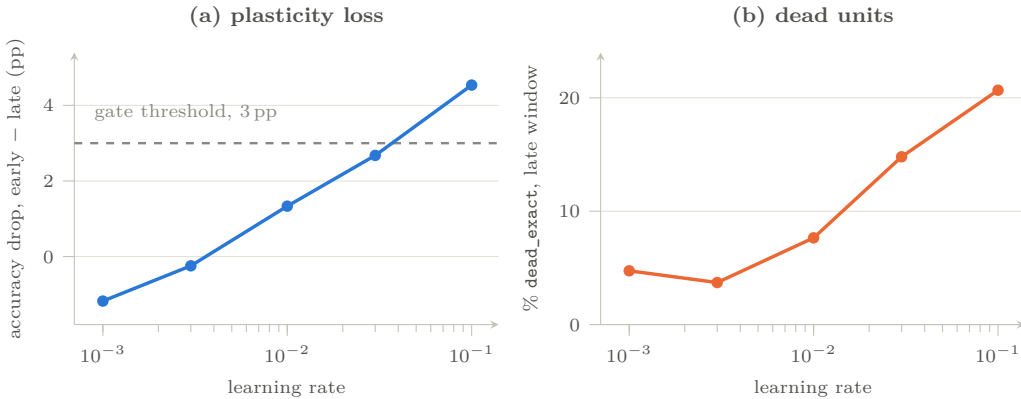


Figure 9: The gate as a dose-response in step size, both criteria. Perfect monotonicity across two decades reproduces Dohare et al.’s finding that the largest step size gives the strongest effect, as a curve rather than at a single point.

Checked before accepting the pass. A diverged run can fake an accuracy drop, so $lr = 0.1$ was audited before use: no NaN anywhere; maximum mean per-task loss 0.472; minimum online accuracy 0.8505; peak 0.9304 at task 1; held-out probe accuracy falling 95.88% → 93.77% independently of the online measure; and no layer wholly dead (maximum 27.9%), so the degenerate 0/0 case in the Sokar score is not driving the dormancy counts. The regime is healthy, not diverged.

Note the asymmetry: the frozen plan contains a stop-rule for “accuracy passes, dead units flat” but the gate produced the mirror image — accuracy fails, dead units rise sharply. No stop-rule covered that, so the ordinary failure response applied. We note it because the asymmetry was not anticipated when the plan was frozen, and the plan was not amended to cover it.

13 Per-layer breakdown of the four definitions

Table 8 gives every definition at every threshold, per hidden layer, on the operative setting. It is here because a pooled dead-unit figure can be produced in two very different ways — units spread evenly across layers, or one layer that has collapsed entirely — and the Sokar score is degenerate in the second case (0/0, defined

as 0), which would mark every unit in a dead layer as τ -dormant at every τ and inflate the pooled number without anything interesting happening. The table rules that out: the three layers carry 27.9%/12.8%/20.9% under DEAD-EXACT, so none is close to collapsed, and the $3.5\times$ spread between definitions in Section 5.2 is present *within every layer separately* rather than being an artefact of pooling across them.

Two secondary observations the pooled table cannot show. First, layer 0 is the deadest layer under every definition, consistent with the survival analysis finding permanent death to be a first-layer phenomenon (Section 5.4). Second, the ordering of definitions is identical in all three layers, which is what makes the disagreement systematic rather than noise: it is not that each definition wins somewhere, but that they are measuring quantities related by a stable, large offset.

Table 8: Every death definition, per hidden layer, at $\text{lr} = 0.1$ over the late window, no intervention, on the current-task probe batch. The two rows in italics are the cross-implementation check required by Section 3: $\text{DORMANT}_\tau[0]$ and DEAD-EXACT are the same quantity computed by two independent code paths and agree to every digit shown, and $\text{DEAD-ABS}_a[10^{-6}]$ should sit just above DEAD-EXACT, which it does.

definition	layer 0	layer 1	layer 2	pooled
DEAD-EXACT	27.88%	12.75%	20.86%	20.67%
$\text{DORMANT}_\tau[0]$ (<i>same quantity</i>)	27.88%	12.75%	20.86%	20.67%
$\text{DEAD-ABS}_a[10^{-6}]$	27.95%	12.78%	20.94%	20.73%
$\text{DEAD-ABS}_a[10^{-4}]$	33.99%	15.65%	27.08%	25.68%
$\text{DEAD-ABS}_a[10^{-2}]$	75.10%	50.77%	67.74%	64.71%
$\text{DORMANT}_\tau[0.01]$	54.41%	31.20%	41.15%	42.43%
$\text{DORMANT}_\tau[0.025]$	64.25%	40.04%	50.17%	51.63%
$\text{DORMANT}_\tau[0.05]$	70.46%	47.33%	57.11%	58.47%
$\text{DORMANT}_\tau[0.1]$ (<i>ReDo's operating point</i>)	75.31%	54.87%	64.16%	64.90%
$\text{DORMANT}_\tau[0.25]$	80.07%	64.42%	72.95%	72.58%
SATURATED	undefined — ReLU is unbounded			
<i>the same DEAD-EXACT, measured on the fixed reference batch instead:</i>				
DEAD-EXACT (reference probe)	26.86%	12.06%	37.15%	26.02%

14 Supporting figures for C2 and C5

Two figures referenced from the main text, held here because their content is already given as tables there — Table 3 for the activation sweep and Table 5 for the optimizer arms — and the figures add the shape of the effect rather than new numbers.

Figure 10 is the graphical form of Section 5.2’s central claim: four definitions, five activation functions, one axis. The bars for LeakyReLU, GELU and SiLU under DEAD-EXACT are absent because they are exactly zero, which is the result rather than a rendering failure.

Figure 11 shows where Adam’s death actually accumulates, which is across tasks rather than within them — the reverse of what Section 5.5’s pre-registered hypothesis predicted.

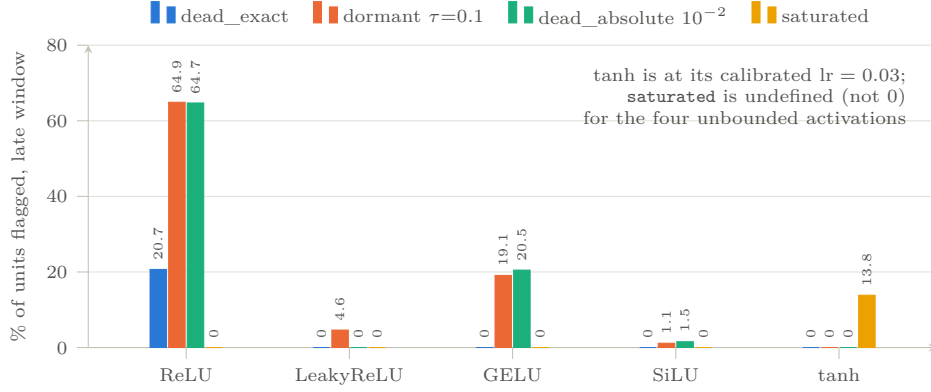


Figure 10: **Four definitions across five activation functions** (Table 3). The figure drops any arm within 5 *pp* of chance and says so on its face, so the diverged tanh-at-lr=0.1 arm cannot reach a plot by accident; the calibrated tanh arm is at lr = 0.03.

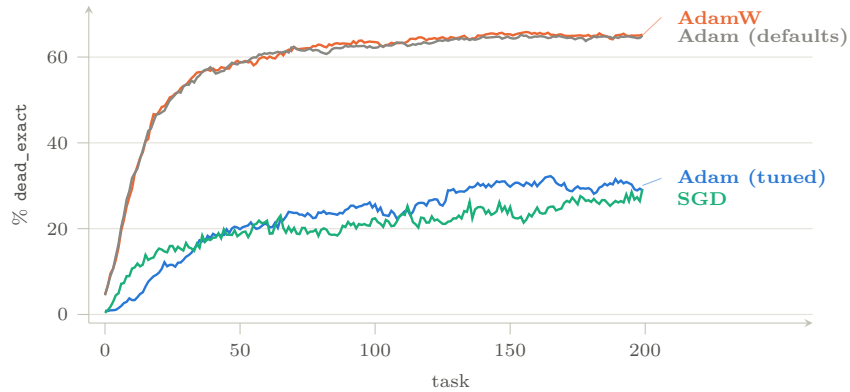


Figure 11: **Where Adam’s death accumulates** (Section 5.5). Adam and AdamW track each other almost exactly, which is the point of including both. The panel showing the pre-registered within-task spike is **not drawn**: the analysis extract for these runs predates a fix to the extractor that omitted the intra-task table, and the figure states this on its face rather than shipping only the panel that worked. The intra-task numbers in Section 5.5 come from the run outputs directly.

15 Compute and reproducibility

Table 9 is the full accounting: 310 runs and 77.71 GPU-hours, summed from each run’s own recorded wall time rather than estimated from the plan. It is given per experiment because the distribution is uneven and informative — the τ -sweep is more than half of it, and the C3 arms cost more than the entire reproduction gate.

Table 9: Compute accounting, summed from each run’s own recorded `gpu_hours`. All runs on a single NVIDIA T4; two independent configurations were run in parallel, one per GPU. The 25 CIFAR-10 runs are excluded — they produced the channel-level measurement in Section 5.2 but their analysis extract is not part of this accounting.

experiment	runs	seeds	GPU-h	purpose
reproduction gate	25	5	5.53	lr ladder, 5 values (§12)
τ -sweep: none + ReDo	70	10	15.21	C1
τ -sweep: size-matched random	60	10	12.57	C1, the control
τ -sweep: inverse-matched	60	10	12.44	C1, opposite extreme
C3 anomaly arms	35	5	19.00	C3
activation sweep	25	5	5.37	C2
tanh lr calibration	15	5	3.37	C2
optimizer arms	20	5	4.21	C5
total	310		77.71	

Logged data. Four tables per run, in Parquet: `tasks` (per task), `metrics` (per task $\times$ layer), `neurons` (per task $\times$ layer $\times$ neuron — the survival analysis of Section 5.4), and `recycling` (per event $\times$ layer — the composition table that Section 5.1 rests on), plus `intra_task` where sub-task probing was enabled. Every metric is logged on both probe batches.

A note on config hashes. Runs are identified by a hash of the resolved configuration. Four fields were added to the configuration schema partway through the study (`data.flatten`, `model.architecture`, `probe.intra_task_probe_every`, `probe.intra_task_probe_reference`), all no-ops for the primary setting. The same experiment therefore hashes differently before and after that change, and completed runs from before it cannot be resumed by the current code. The bit-identical reproduction of those runs under the new code is the evidence that behaviour did not change; we state it here rather than leave a reader to discover two hashes for one experiment.

Reproducibility statement

All experiments are deterministic and were verified bit-reproducible across machines and accounts. The analysis plan was frozen and timestamped before data collection and is reproduced verbatim in Section 11. Every number in this paper is generated from the committed parquet outputs by a single script that applies the frozen estimator, and every data figure is generated as $\text{\LaTeX}$ from those same outputs; none is transcribed by hand. Section 15 gives the full compute accounting.

Acknowledgments

To be added.

References

- Zaheer Abbas, Rosie Zhao, Joseph Modayil, Adam White, and Marlos C. Machado. Loss of plasticity in continual deep reinforcement learning. *Conference on Lifelong Learning Agents (CoLLAs)*, 2023. arXiv:2303.07507.
- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.
- Jordan T. Ash and Ryan P. Adams. On warm-starting neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:1910.08475.
- Tudor Berariu, Wojciech Czarnecki, Soham De, Jorg Bornschein, Samuel Smith, Razvan Pascanu, and Claudia Clopath. A study on the plasticity of neural networks. *arXiv preprint arXiv:2106.00042*, 2021.
- Vitaliy Chiley, Ilya Sharapov, Atli Kosson, Urs Koster, Ryan Reece, Sofia Samaniego de la Fuente, Vishal Subbiah, and Michael James. Online normalization for training neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1905.05894.
- Shibhansh Dohare, Richard S. Sutton, and A. Rupam Mahmood. Continual backprop: Stochastic gradient descent with persistent randomness. *arXiv preprint arXiv:2108.06325*, 2021.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, 2024. doi: 10.1038/s41586-024-07711-7.
- Simon Dufort-Labbé, Pierluca D’Oro, Evgenii Nikishin, Razvan Pascanu, Pierre-Luc Bacon, and Aristide Baratin. Maxwell’s demon at work: Efficient pruning by leveraging saturation of neurons. *Transactions on Machine Learning Research*, 2025. arXiv:2403.07688.
- Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.
- Mohamed Elsayed and A. Rupam Mahmood. Addressing loss of plasticity and catastrophic forgetting in continual learning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2404.00781.
- Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- Caglar Gulcehre, Srivatsan Srinivasan, Jakub Sygnowski, Georg Ostrovski, Mehrdad Farajtabar, Matt Hoffman, Razvan Pascanu, and Arnaud Doucet. An empirical study of implicit regularization in deep offline reinforcement learning. *Transactions on Machine Learning Research*, 2022. arXiv:2207.02099.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.
- Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- J. Fernando Hernandez-Garcia, Shibhansh Dohare, Jun Luo, and Richard S. Sutton. Reinitializing weights vs units for maintaining plasticity in neural networks. *arXiv preprint arXiv:2508.00212*, 2025.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

-
- Timo Klein, Christoph Luther, Manus McAuliffe, Lukas Miklautz, Claudia Plant, and Sebastian Tschiatschek. Plasticity loss in deep reinforcement learning: A survey. *arXiv preprint arXiv:2411.04832*, 2024.
- Jacob E. Kooi, Zhao Yang, Mark Hoogendoorn, and Vincent François-Lavet. Hadamard representations: Augmenting hyperbolic tangents in rl. *arXiv preprint arXiv:2406.09079*, 2024.
- Aviral Kumar, Rishabh Agarwal, Dibya Ghosh, and Sergey Levine. Implicit under-parameterization inhibits data-efficient deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.14498.
- Saurabh Kumar, Henrik Marklund, and Benjamin Van Roy. Maintaining plasticity in continual learning via regenerative regularization. *arXiv preprint arXiv:2308.11958*, 2023.
- Alex Lewandowski, Haruto Tanaka, Dale Schuurmans, and Marlos C. Machado. Directions of curvature as an explanation for loss of plasticity. *arXiv preprint arXiv:2312.00246*, 2024.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Lu Lu, Yeonjong Shin, Yanhui Su, and George Em Karniadakis. Dying ReLU and initialization: Theory and numerical examples. *Communications in Computational Physics*, 2020. arXiv:1903.06733.
- Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, 2023. arXiv:2303.01486.
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, Hado van Hasselt, Razvan Pascanu, James Martens, and Will Dabney. Disentangling the causes of plasticity loss in neural networks. In *Conference on Lifelong Learning Agents (CoLLAs)*, 2024. arXiv:2402.18762.
- Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013.
- Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010.
- Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022. arXiv:2205.07802.
- Evgenii Nikishin, Junhyuk Oh, Georg Ostrovski, Clare Lyle, Razvan Pascanu, Will Dabney, and André Barreto. Deep reinforcement learning with plasticity injection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.15555.
- Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *15th European Signal Processing Conference (EUSIPCO)*, pp. 606–610, Poznań, Poland, 2007.
- Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, 2023. arXiv:2302.12902.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56): 1929–1958, 2014.
- Elena Voita, Javier Ferrando, and Christoforos Nalmpantis. Neurons in large language models: Dead, n-gram, positional. In *Findings of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2309.04827.
- Tim Whitaker and Darrell Whitley. Synaptic stripping: How pruning can bring dead neurons back to life. *arXiv preprint arXiv:2302.05818*, 2023.